Tecsup
TECNOLOGÍA
CON SENTIDO

INFORME DE LABORATORIO 11

Firebase



PROGRAMACIÓN EN MÓVILES

Docente: Benjamin Pareja

Programa: Diseño y Desarrollo de Software

2025 - Sur



Alumno(s):	PUMA LUCHO NICOLAS LIAM	***Nota***	
Grupo:		***Ciclo:***	

Criterio de Evaluación lab 11 android	*Excelente (4pts)*	*Bueno (3pts)*	*Requiere mejora (2pts)*	*No acept. (0pts)*	*Puntaje Logrado*
Sube commits por cada avance del laboratorio					*4*
Configuración de Firebase					*4*
Desarrolla la parte guiada del laboratorio y el ejercicio					*6*
Realiza la pregunta de observación					*3*
Es puntual y redacta el informe adecuadamente sin copias de otros autores					*3*

Laboratorio 11: Firebase

I. Objetivo

- Conocer Firebase
- Emplear algunos componentes de Firebase en Android con Jetpack Compose

II. Seguridad

- No ingresar con líquidos, ni comida al aula de Laboratorio.
- Al culminar la sesión de laboratorio apagar correctamente la computadora y la pantalla, y ordenar las sillas utilizadas.

III. Equipos y Materiales

- Sistema Operativo Windows 10 o superior con conexión a la red del laboratorio
- Android Studio o algún otro entorno de desarrollo donde se pueda correr aplicaciones móviles basadas en kotlin y jetpack compose .

DESARROLLO

Para la calificación de la parte guiada se considerará como puntaje que creen el repositorio de este laboratorio, y que realicen commits por cada cambio/nuevo archivo/modificación que realicen, siguiendo las buenas prácticas vistas en el laboratorio 4.

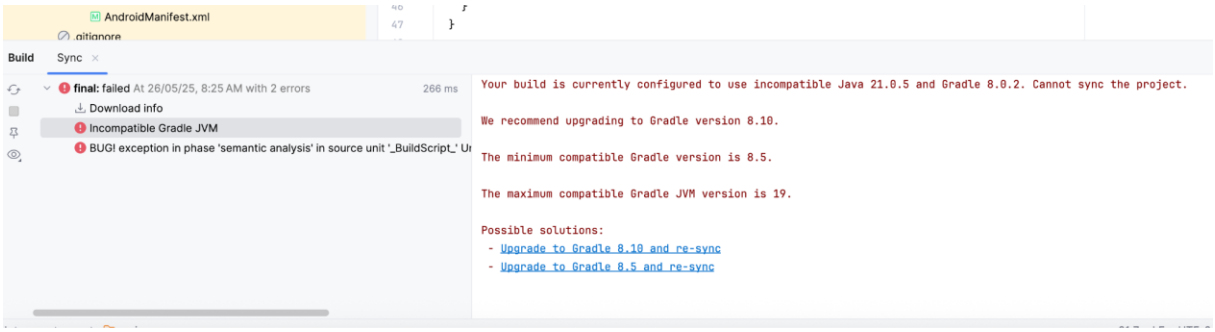
A partir de este laboratorio se solicita compartir también un video corto de los ejercicios funcionando, algo como esto:

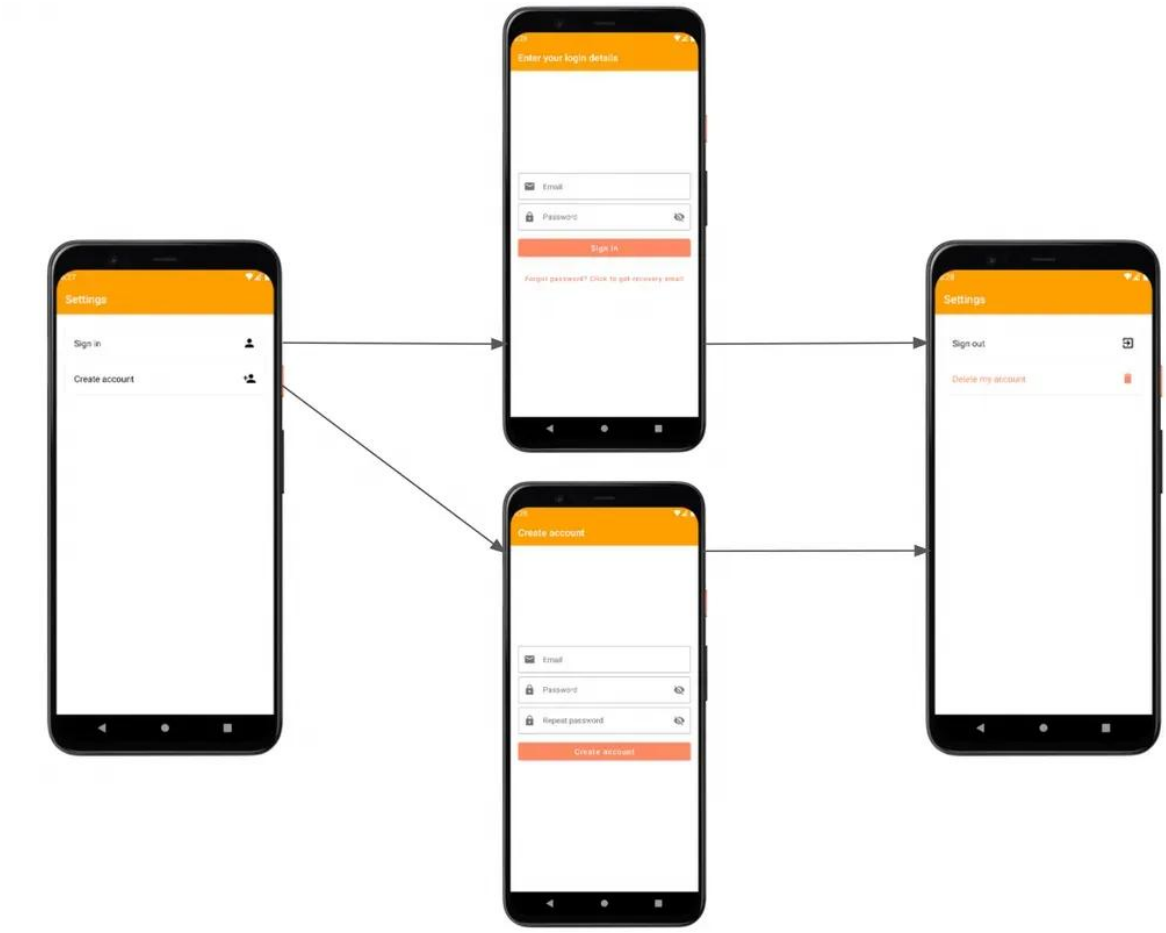
ScreenRecording.mp4

1. Clonar el repositorio de github y abre el proyecto en android studio:

https://github.com/BparejaTec/Lab11_Start

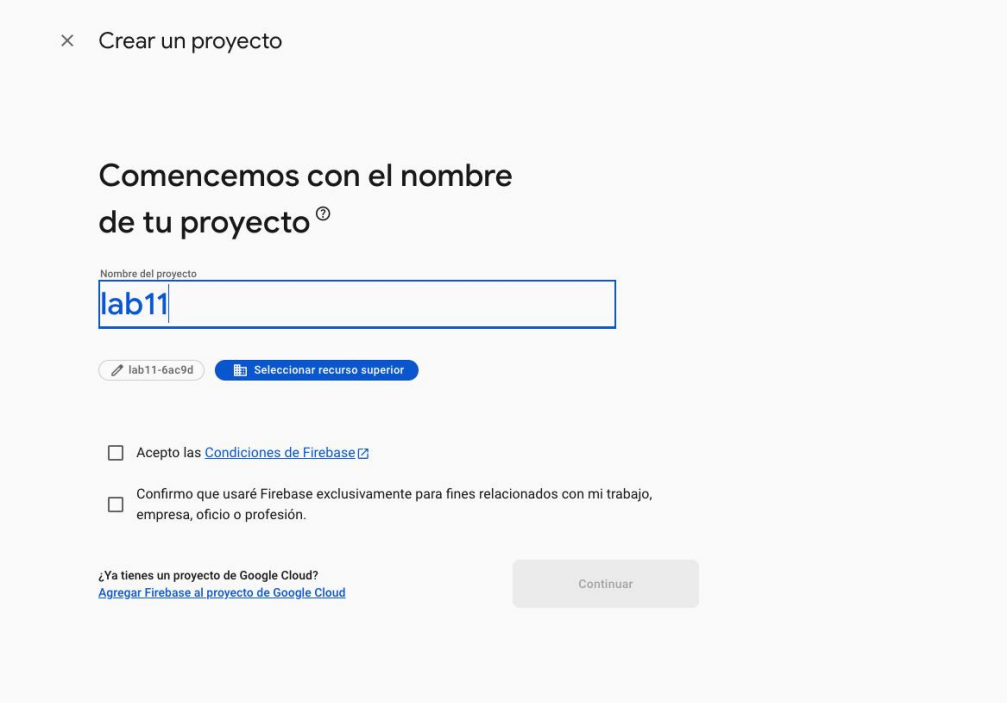
(Si les aparece un error similar, pueden actualizar a gradle 8.10)





2. Configurar Firebase

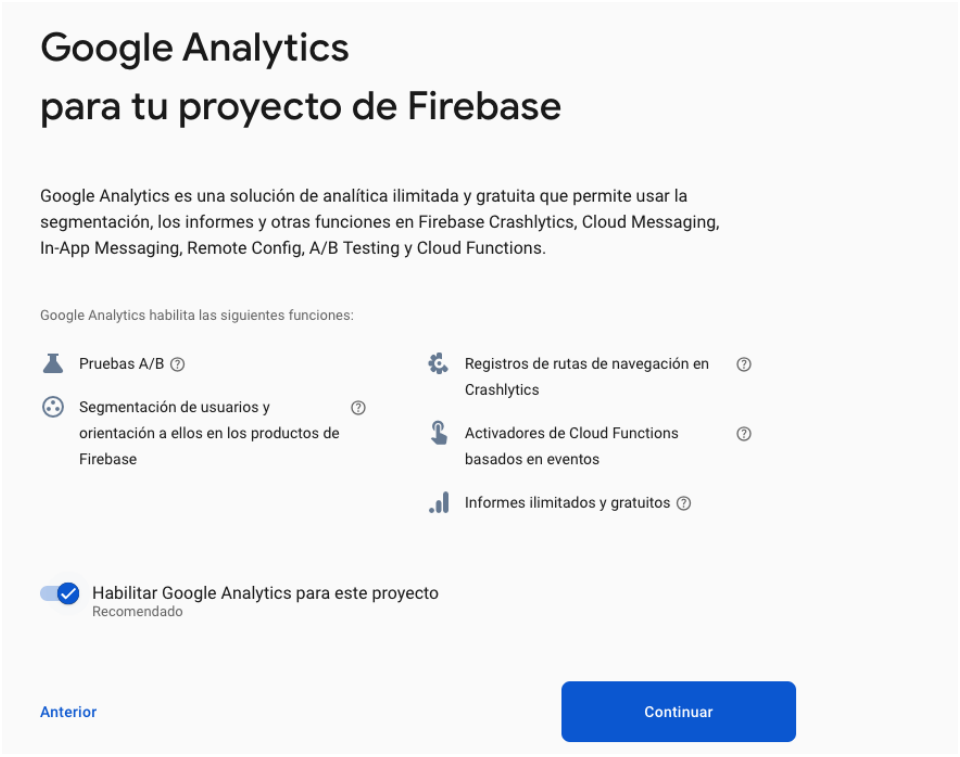
Lo primero que necesitas hacer es ir a la consola de Firebase y crear un proyecto de Firebase haciendo clic en el botón "+ Add project", como puedes ver a continuación:



Firebase console

Sigue los pasos en la pantalla para completar la creación del proyecto.

Nota: Por favor, habilita Google Analytics al crear el proyecto. Habilitar Google Analytics proporciona opciones expandidas de segmentación en Firebase Remote Config.



Dentro de cada proyecto de Firebase, puedes crear diferentes aplicaciones: para Android, iOS, Web, Flutter y Unity. Elige la opción Android, como se ve aquí:



Firebase Project Overview

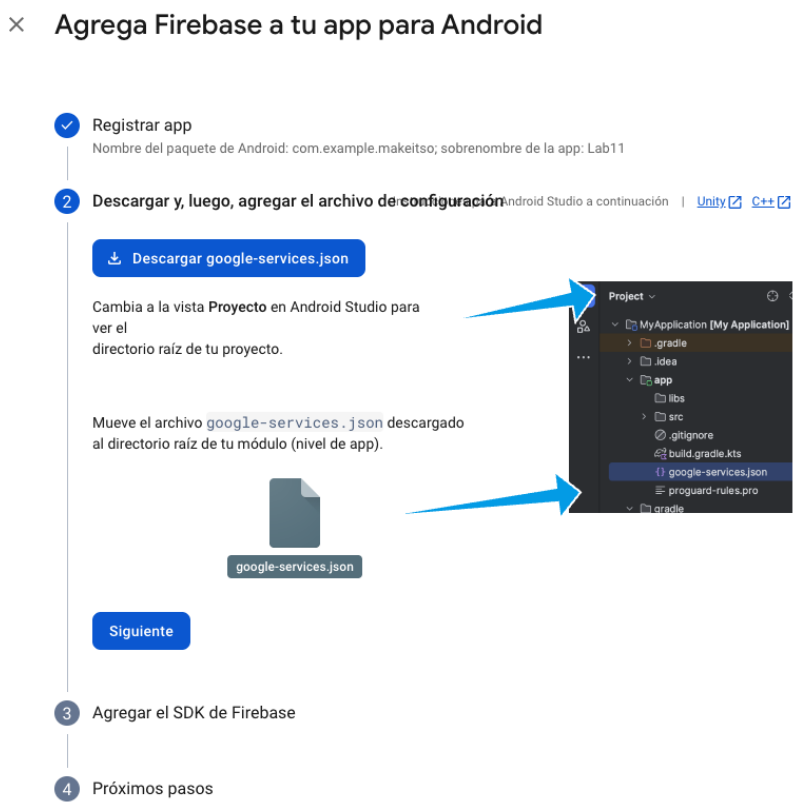
Luego sigue estos pasos:

1. Ingresa `com.example.makeitso` como el nombre del paquete y, opcionalmente, ingresa un apodo, no necesitas añadir el certificado de firma de depuración.



2. Haz clic en **Next** para registrar tu aplicación y acceder al archivo de configuración de Firebase.

3. Haz clic en **Download google-services.json** para descargar tu archivo de configuración y guárdalo en el directorio `make-it-so-android/app`.

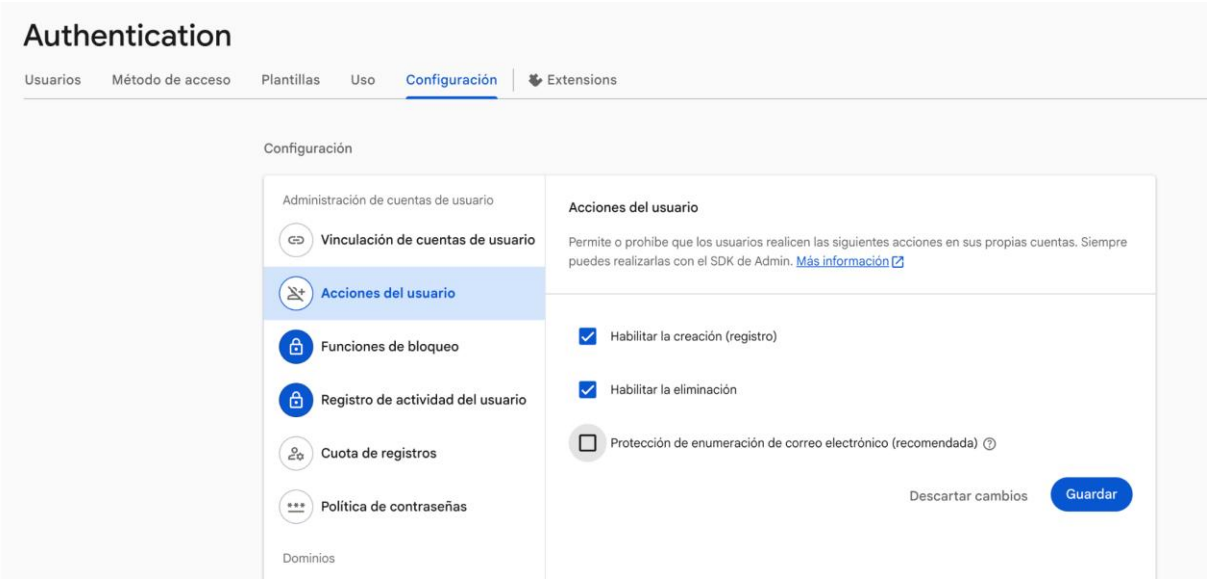


4. Haz clic en **Next**. Dado que los SDKs de Firebase ya están incluidos en el archivo `build.gradle` en el proyecto de ejemplo, haz clic en **Next** para saltar a los pasos finales.
5. Haz clic en **Continue to console** para finalizar.

Para que la aplicación "Make it So" funcione correctamente, hay dos cosas que debes hacer en la consola antes de pasar al código: habilitar proveedores de autenticación y crear la base de datos Firestore.

Primero, habilitamos la Autenticación para que los usuarios puedan iniciar sesión en la aplicación:

1. Desde el menú **Build**, selecciona **Authentication**, y luego haz clic en **Get Started**.
2. Desde la tarjeta **Sign-in method**, selecciona **Email/Password**, y habilítalo.
3. A continuación, haz clic en **Add new provider**, selecciona y habilita **Anonymous**.
4. **IMPORTANTE**: Se debe desactivar en la configuración la “Protección de enumeración de correo electrónico”, para que la aplicación pueda funcionar luego



---

3. Configurar Firestore

Usarás Firestore para almacenar las tareas de los usuarios que hayan iniciado sesión. Cada usuario tendrá su propio documento dentro de una colección de la base de datos.

1. Desde el menú **Build**, selecciona **Firestore**, luego haz clic en **Create database**.
2. Mantén habilitada la opción **Start in production mode** y haz clic en **Next**.
3. Cuando se te pida, selecciona la ubicación donde se almacenarán tus datos de Cloud Firestore. Al desarrollar una aplicación de producción, querrás que esta ubicación esté cerca de la mayoría de tus usuarios y en común con otros servicios de Firebase, como Functions. Para este proyecto, puedes mantener la región predeterminada o seleccionar la región más cercana a ti.
4. Haz clic en **Enable** para provisionar tu base de datos Firestore.

---

4. Establecer reglas de seguridad en Firestore

Es importante construir reglas de seguridad robustas para la base de datos Firestore. Abre el panel de Firestore y ve a la pestaña **Rules**. Luego, actualiza las reglas de seguridad para que se vean así:

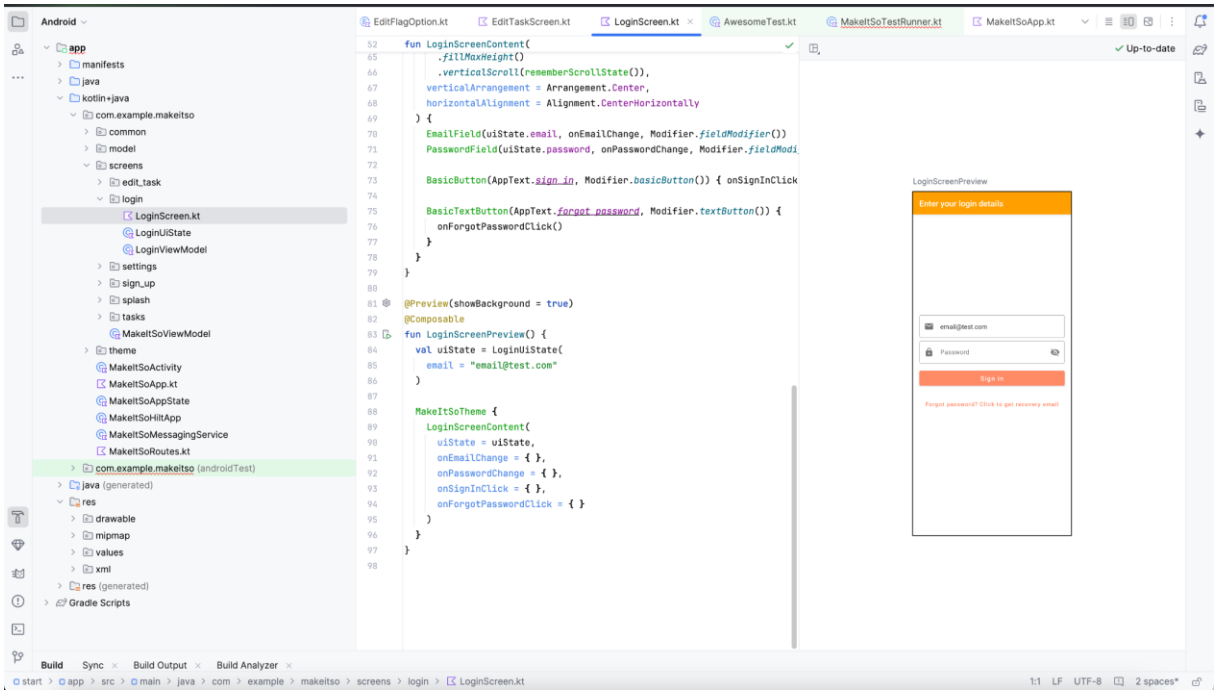
```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /{document=**} {
      allow create: if request.auth != null;
      allow read, update, delete: if request.auth != null &&
resource.data.userId == request.auth.uid;
    }
  }
```

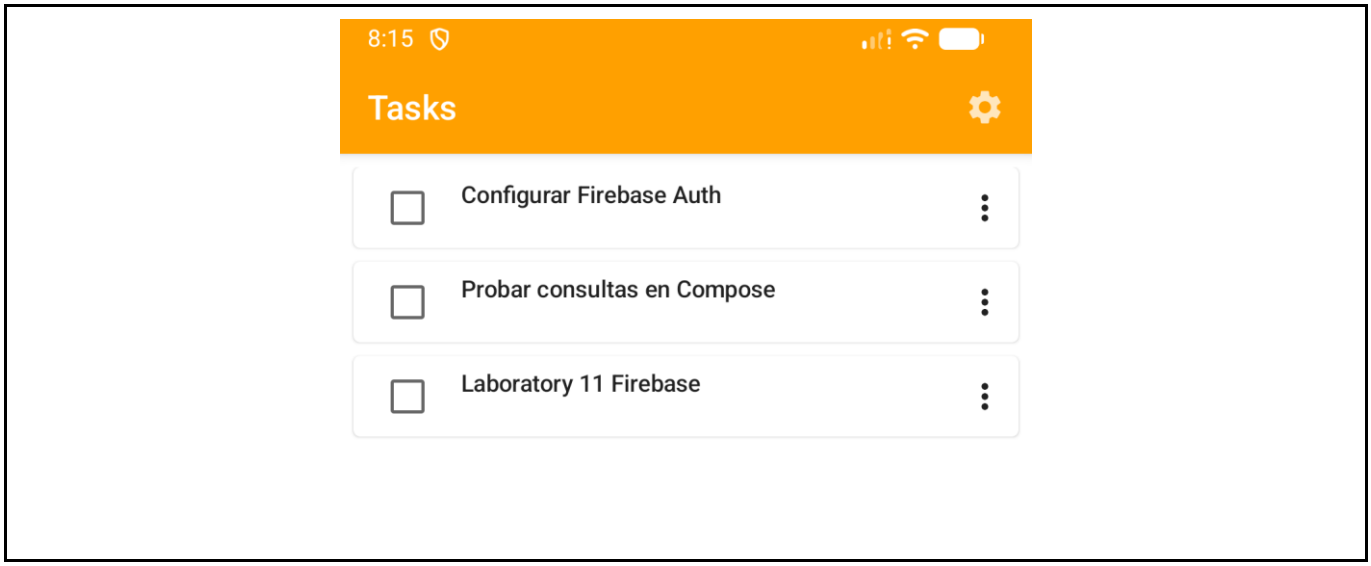
```
}
```

Estas reglas básicamente dicen que cualquier usuario que haya iniciado sesión en la aplicación puede crear un documento para sí mismo dentro de cualquier colección. Luego, una vez creado, solo el usuario que creó ese documento podrá verlo, actualizarlo o eliminarlo.

Ejecutar la aplicación

Ahora estás listo para ejecutar la aplicación. Abre la carpeta `make-it-so-android/start` en Android Studio y ejecuta la aplicación (puede hacerse usando un emulador de Android o un dispositivo Android real).





Crear autentificación para la aplicación

En el estado actual de la aplicación de ejemplo **Make It So**, un usuario puede comenzar a usar la aplicación sin necesidad de iniciar sesión primero. Utiliza la autenticación anónima para lograr esto. Sin embargo, las cuentas anónimas no permiten que un usuario acceda a sus datos en otros dispositivos o incluso en sesiones futuras. Aunque la autenticación anónima es útil para una incorporación rápida, siempre deberías proporcionar la opción para que los usuarios conviertan su cuenta a otro tipo de inicio de sesión. Con esto en mente, en este proyecto agregarás autenticación por correo electrónico y contraseña a la aplicación **Make It So**.

Tan pronto como el usuario cree una cuenta escribiendo un correo electrónico y una contraseña, necesitas pedirle a la API de Firebase Authentication un **email credential**, y luego vincular la nueva credencial a la cuenta anónima. Abre el archivo **AccountServiceImpl.kt** en Android Studio y actualiza la función `linkAccount` para que se vea como el siguiente código:

model/service/impl/AccountServiceImpl.kt

kotlin

```
override suspend fun linkAccount(email: String, password: String) {
    val credential = EmailAuthProvider.getCredential(email, password)
    auth.currentUser!!.linkWithCredential(credential).await()
}
```

Ahora abre **SignUpViewModel.kt** y llama a la función `linkAccount` del servicio dentro del bloque `launchCatching` de la función `onSignUpClick`:

screens/sign_up/SignUpViewModel.kt

```
launchCatching {
    accountService.linkAccount(email, password)
    openAndPopUp(SETTINGS_SCREEN, SIGN_UP_SCREEN)
}
```

Primero intenta autenticar, y si la llamada tiene éxito, procede a la siguiente pantalla (la **SettingsScreen**). Como estás ejecutando estas llamadas dentro de un bloque `launchCatching`, si ocurre un error en la primera línea, la excepción será capturada y manejada, y la segunda línea no se alcanzará en absoluto

---

Actualización de la pantalla de configuración

Tan pronto como la **SettingsScreen** se vuelva a abrir, debes asegurarte de que las opciones de **Sign in** y **Create account** desaparezcan, porque ahora el usuario ya está autenticado. Para hacer esto, vamos a hacer que el **SettingsViewModel** escuche el estado del usuario actual (disponible en **AccountService.kt**) para verificar si la cuenta es anónima o no. Para ello, actualiza el `uiState` en **SettingsViewModel.kt** para que se vea así:

screens/settings/SettingsViewModel.kt

```
val uiState = accountService.currentUser.map {
    SettingsUiState(it.isAnonymous)
}
```

Lo último que necesitas hacer es actualizar el `uiState` en **SettingsScreen.kt** para recolectar los estados emitidos por el **SettingsViewModel**:

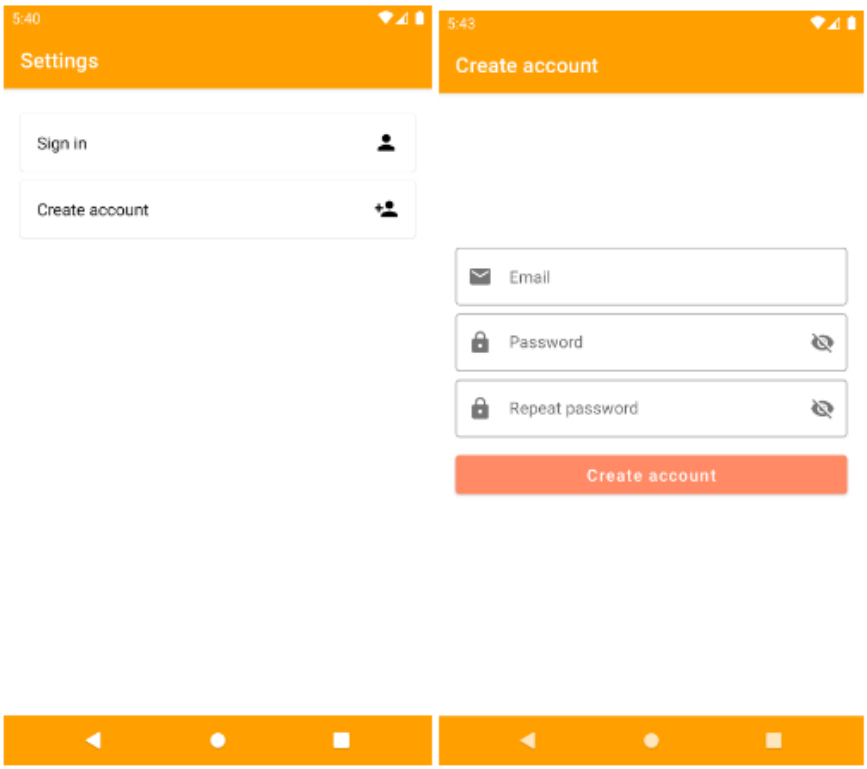
screens/settings/SettingsScreen.kt

```
val uiState by viewModel.uiState.collectAsState(
    initial = SettingsUiState(false)
)
```

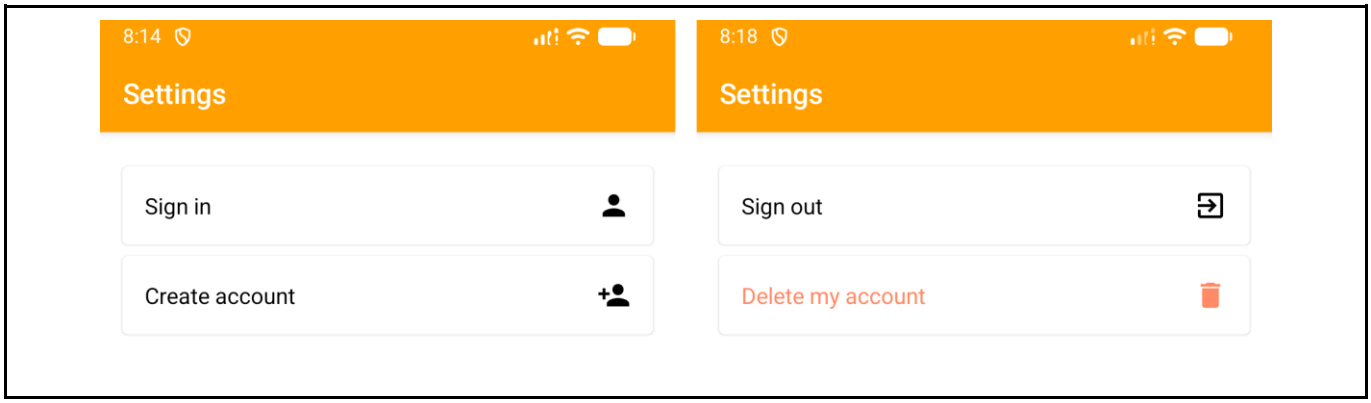
Ahora, cada vez que el usuario cambie, la **SettingsScreen** se volverá a componer para mostrar las opciones de acuerdo con el nuevo estado de autenticación del usuario.

Ejecutar la aplicación

Ejecuta **Make It So** y navega a la configuración haciendo clic en el ícono de engranaje en la esquina superior derecha de la pantalla. Desde allí, haz clic en la opción **create account**:



Escribe un correo electrónico válido y una contraseña fuerte para crear tu cuenta. Debería funcionar y deberías ser redirigido a la página de configuración, donde verás dos nuevas opciones: **sign out** y **delete your account**. Puedes verificar la nueva cuenta creada en el panel de **Authentication** en la consola de Firebase haciendo clic en la pestaña **Users**.



Cloud Firestore

Para Cloud Firestore, agregarás un listener a la colección de Firestore que almacena los documentos que representan las tareas mostradas en **Make It So**. Una vez que agregues este listener, recibirás cada actualización realizada en esta colección.

Actualiza el **Flow** disponible en **StorageServiceImpl.kt** para que se vea así:

model/service/impl/StorageServiceImpl.kt

```
override val tasks: Flow<List<Task>>
    get() =
      auth.currentUser.flatMapLatest { user ->
        firestore.collection(TASK_COLLECTION).whereEqualTo(USER_ID_FIELD,
user.id).dataObjects()
      }
```

Este código está agregando un listener a la colección `tasks` basada en el `user.id`. Cada tarea está representada por un documento en una colección llamada `tasks`, y cada una de ellas tiene un campo llamado `userId`. Ten en cuenta que se emitirá un nuevo **Flow** si cambia el estado de `currentUser` (por ejemplo, al cerrar sesión).

Ahora necesitas hacer que el **Flow** en **TasksViewModel.kt** refleje lo mismo que en el servicio:

screens/tasks/TasksViewModel.kt

```
val tasks = storageService.tasks
```

Lo último será hacer que la función **composable** en **TasksScreens.kt**, que representa la interfaz de usuario, sea consciente de este flujo y lo recoja como un estado. Cada vez que cambie el estado, la función **composable** se volverá a componer automáticamente y mostrará el estado más reciente al usuario. Añade esto a la función composable **TasksScreen**:

screens/tasks/TasksScreen.kt

```
val tasks = viewModel
    .tasks
    .collectAsStateWithLifecycle(emptyList())
```

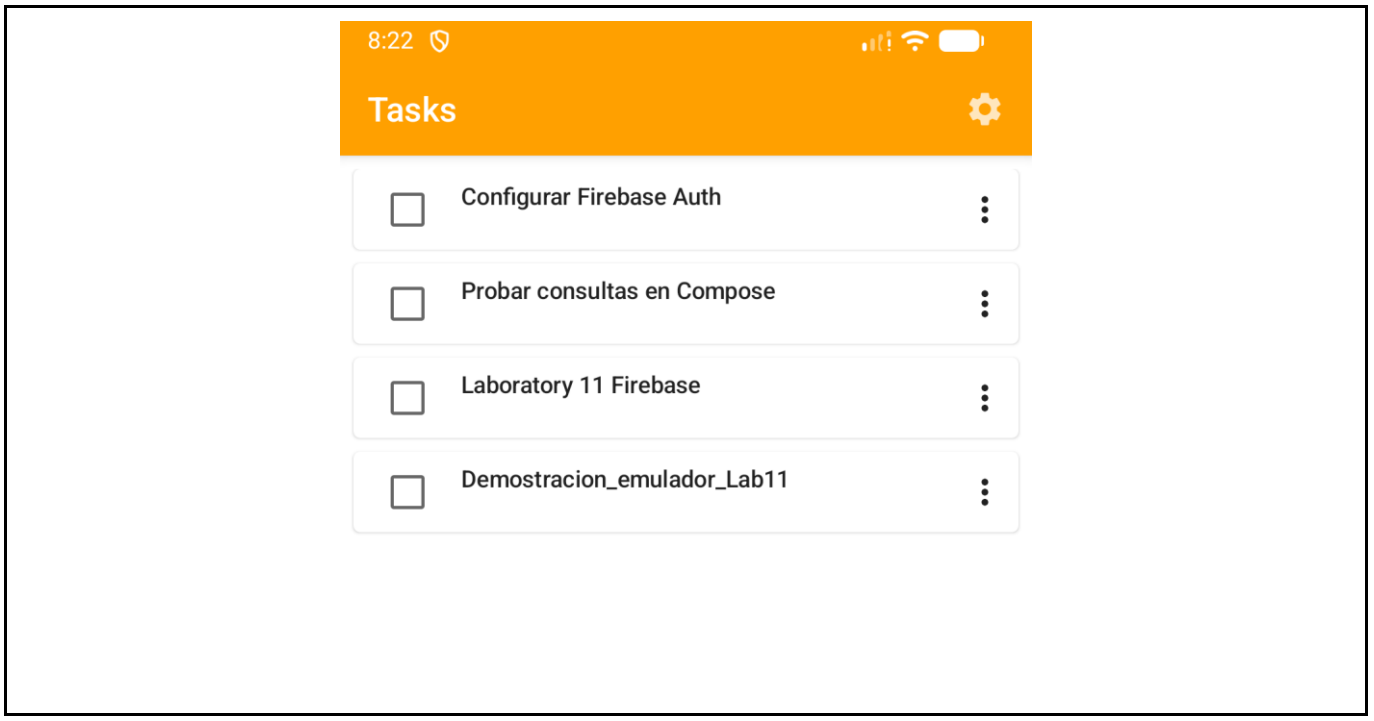
Una vez que la función composable tenga acceso a estos estados, puedes actualizar el **LazyColumn** (que es la estructura que usas para mostrar una lista en la pantalla) para que se vea así:

screens/tasks/TasksScreen.kt

```
LazyColumn {
    items(tasks.value, key = { it.id }) { taskItem ->
        TaskItem( [...] )
    }
}
```

Ejecutar la aplicación

Para comprobar que funciona, agrega una nueva tarea usando la aplicación (haciendo clic en el botón de agregar en la esquina inferior derecha de la pantalla). Una vez que termines de crear la tarea, debería aparecer en la colección de Firestore en la consola de Firestore. Si inicias sesión en **Make It So** en otros dispositivos con la misma cuenta, podrás editar tus tareas y ver cómo se actualizan en todos los dispositivos en tiempo real.



Ejercicio de desarrollo

En caso uses firebase en tu proyecto, menciona que componentes usas, muestra el desarrollo que hiciste de momento para que funcione.

(comparte un video del funcionamiento)

Componentes de Firebase que usé en la app

Authentication: la app arranca con autenticación anónima y desde Settings vinculo esa cuenta con correo y contraseña. Con eso el usuario pasa a tener una cuenta real sin perder las tareas que ya había creado.

Cloud Firestore: guardo cada tarea como un documento de la colección tasks, con el campo userId. La consulta filtra con whereEqualTo(USER_ID_FIELD, user.id), así cada usuario ve solamente sus tareas.

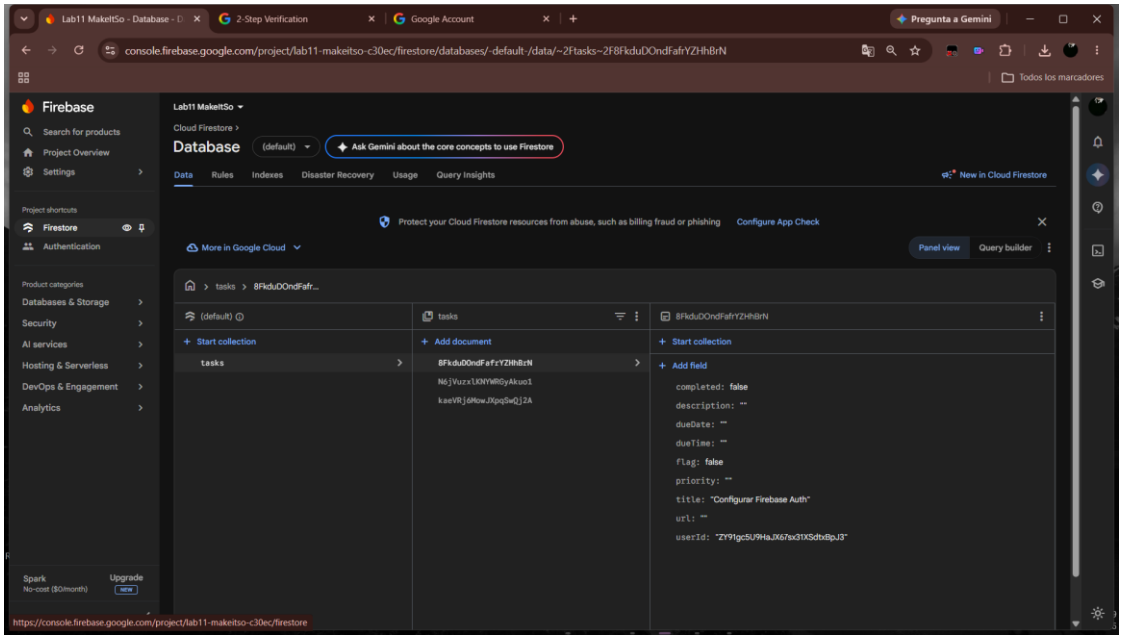
Analytics: se habilitó al crear el proyecto y queda registrando los eventos de la aplicación.

Performance Monitoring: agregué la función trace() en Performance.kt y la usé en linkAccount, saveTask y updateTask para medir cuánto demoran esas operaciones.

Reglas de Firestore: permiten crear a cualquier usuario autenticado y dejan leer, actualizar o borrar solo cuando resource.data.userId es igual a request.auth.uid.

Repositorio del laboratorio: https://github.com/NicolasPumaLuicho/Lab11-MakeItSo

Video del funcionamiento: ScreenRecording.mp4 (adjunto en la entrega).



Puntos adicionales

Avanza por lo menos hasta el punto 6 del proyecto planteado en los artículos de firebase y jetpack compose y comparte la aplicación funcionando aquí. (3 ptos)

1 / Overview of how this app is structured.

2 / Implement login and sign up flows using Firebase Authentication.

3 / Add Firebase Crashlytics to handle errors.

4 / Store data in the cloud using Cloud Firestore.

5 / Add Kotlin Coroutines and Kotlin Flow.

6 / Improve the app's quality with Firebase Performance Monitoring.

7 / Control app behavior and appearance remotely using Firebase Remote Config.

8 / Add new features to your app with little effort with Firebase Extensions.

9 / Send push notifications with Firebase Cloud Messaging.

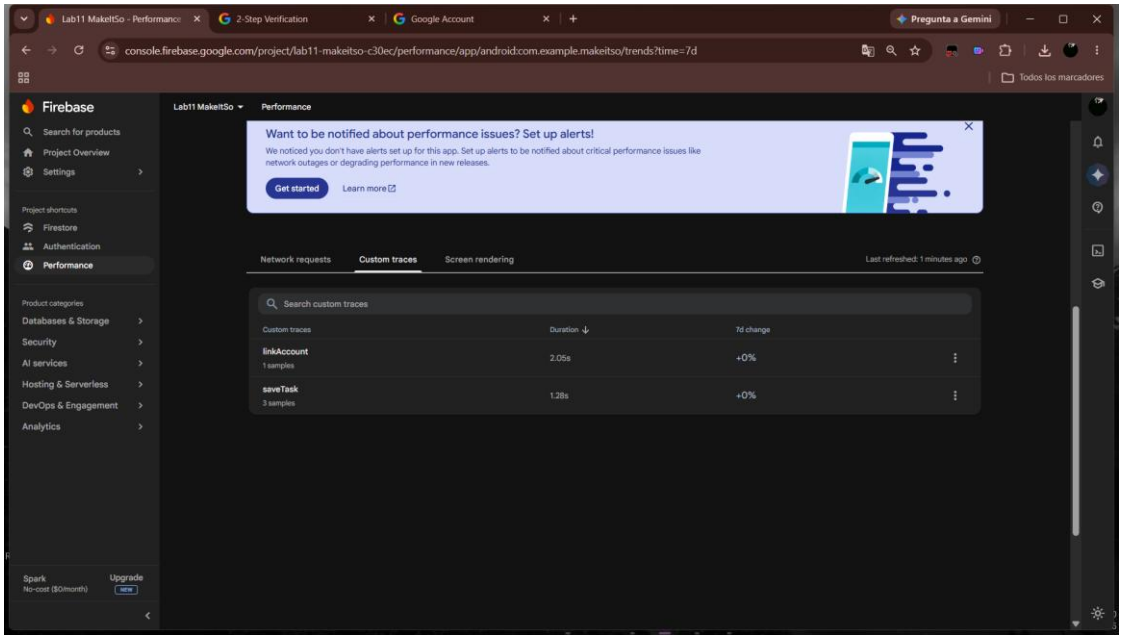
(comparte un video del funcionamiento)

Puntos del artículo cubiertos

Avancé hasta el punto 6 del proyecto del artículo, que es mejorar la calidad de la app con Firebase Performance Monitoring. Los puntos anteriores quedan cubiertos con el proyecto Make It So: la estructura general (1), el login y registro con Firebase Authentication (2), Crashlytics incluido en el proyecto (3), Cloud Firestore para guardar las tareas (4) y Kotlin Coroutines con Flow para que la lista se actualice sola (5).

Para el punto 6 usé trace() en las tres operaciones que más pesan: linkAccount, saveTask y updateTask. En la consola, dentro de Performance y en la pestaña Custom traces, se ven los tiempos de las trazas: linkAccount con 2.05 s y saveTask con 1.28 s.

Video del funcionamiento: ScreenRecording.mp4 (adjunto en la entrega).



PREGUNTA DE OBSERVACIÓN:

- Selecciona una parte del laboratorio que te haya resultado más desafiante. Explica por qué fue difícil y qué hiciste para resolverlo.

  La parte que más me costó fue dejar el proyecto compilando. Al sincronizar me salió un error de Java incompatible con Gradle 8.0.2 porque la máquina tenía Java 25. Leí el mensaje, instalé el JDK 17 y compilé apuntando a esa versión con JAVA_HOME, y recién ahí pude instalar la app en el emulador. También me demoré con las reglas de Firestore, porque cuando la regla de lectura no comparaba resource.data.userId con request.auth.uid la lista de tareas aparecía vacía.

- Explica cómo usaste la IA para el desarrollo de este laboratorio.

  Usé la IA como guía durante casi todo el laboratorio: me ayudó a crear el proyecto en Firebase, a entender el error de Java y Gradle, a escribir los cambios en Kotlin y a ordenar las reglas de Firestore. Yo iba revisando cada paso en el emulador, probando que las tareas se guardaran y aparecieran en la consola de Firebase, y tomando las capturas que están en el informe.